



The University of Texas Rio Grande Valley
College of Engineering and Computer Science
Department of Electrical & Computer Engineering

CSCI-3334-01: Operating Systems
Spring 2026

Final Project Report

By

Name: Edward Garza

Student ID: 20592635

Instructor: Satzhan Sitmukhambetov

Date: 5/06/2026

I.	Introduction	2
II.	Architecture	2
III.	Synchronization Strategy	2
IV.	Scheduling Policy	2
V.	Metrics Collected	3
VI.	Experiment Results	3
VII.	Analysis of Results	4
VIII.	Bug or Design Problems Encountered	4
IX.	Conclusion	4

I. Introduction

For this Final Project, we are programming a concurrent task dispatcher within Rust. It will simulate a stream of bound tasks like CPU and IO. These types of tasks will be created over time as well as being placed into a shared queue within the program and once out of the queue it will be dispatched to a bounded pool of worker threads that are in the scheduling policy. When doing this final project, our main objective is to show that this program for this final project can be concurrency, synchronization, scheduling, and performance analysis.

II. Architecture

Within this program we have four important parts that make this program unique:

- Generator Thread: This thread will create the tasks and send them to the dispatcher.
- Dispatcher Thread: This thread will store the tasks in a ready queue, and it will decide whether a task can be assigned to worker thread, but it will base it on CPU availability.
- Worker Pool: This will contain about eight worker threads that will execute tasks within the program until it gets shutdown.
- Monitor Thread: This thread samples queue length and CPU usage with every 10 milliseconds to record the system's behavior during the simulation.

III. Synchronization Strategy

The strategy going into this project was to create a standard program using the Rust Library of concurrency tools. This will include all types of tools like threads, channels, Arc, and Mutex. The reason for having the channels is because they are used to send tasks from the generator to the dispatcher and from the dispatcher to the workers. We are also using the shared state because it will include a ready queue, CPU usage, performance metrics, and CPU statistics. The reason being is for we can track and see if anything is out balanced by one or the other in statistics and see what needs to be changed within the code. As will these values are protected by Arc<Mutex<...>> so that multiple threads can safely access and update them as throughout the code.

IV. Scheduling Policy

FIFO Scheduler: This scheduler will dispatch the tasks in the arrival order, if it is sending the next task that would not cause the global CPU usage to exceed 100%. Optimized IO Priority Scheduler: This scheduler will give the preferences to the IO-bound tasks before the CPU-bound task and when selecting the next task from the queue. This approach is intended to help improve responsiveness for lighter tasks and will also reduce the queue buildup under the mixed workloads within the program.

V. Metrics Collected

With the program results, we will be looking at the following metrics:

- total tasks completed
- number of CPU tasks completed
- number of IO tasks completed
- total runtime
- average CPU usage
- average wait time
- average turnaround time

These are the metrics that will be getting results for this program

VI. Experiment Results

```
===== Final Metrics =====  
Total tasks completed: 1000  
CPU tasks completed: 304  
IO tasks completed: 696  
Total runtime: 27.04s  
Average CPU usage: 88.16%  
Average wait time: 3507 ms  
Average turnaround time: 3708 ms
```

Figure 1: FIFO Results

```
===== Final Metrics =====  
Total tasks completed: 1000  
CPU tasks completed: 304  
IO tasks completed: 696  
Total runtime: 27.04s  
Average CPU usage: 88.22%  
Average wait time: 3510 ms  
Average turnaround time: 3711 ms
```

Figure 2: Optimized Results

VII. Analysis of Results

As you can see in Figure 1 and Figure 2, the FIFO and the IO-priority schedulers are doing as expected, which is to complete all 1000 tasks with similar CPU utilization, runtime, wait time, and turnaround time. However, the results are nearly identical results under the tested workload. So that means either our CPU cap or the worker pool were the major factors affecting the performance during this final project while the scheduling policy has only a small impact. The scheduler was intended to improve the responsiveness of the program by mainly using the IO-Bounded tasks. However, the measured results showed only negligible differences. Like how we said earlier that the workload mixes and resources constraints are reducing task reordering within the simulation.

VIII. Bug or Design Problems Encountered

When working on this program, there were several issues that we encountered. One problem was the rust programming and the moved-value errors inside the thread closures but mostly within the dispatcher. The way it was resolved was to make the structure of the dispatcher much simpler to make sure the channels and shared state are moved to the correct thread once only. Another issue that was encountered was the CPU accounting because the CPU usage was updated sometime after or too late from the program. This allowed the dispatcher to overschedule tasks and caused the CPU usage to exceed 100%. The way to fix it was to reserve the CPU capacity in the dispatcher before it sends the tasks to the workers. As well another issue that was presented was that the optimized IO-priority scheduler was implemented into our program, the results were very similar to the FIFO. This means it will show the resource limits that had a stronger effect on the performance than the scheduling order in the workload. The final issue that was encountered in this project was a bug that occurred when the CPU usage was subtracted incorrectly at task completion. This would cause an unsigned integer overflow and thread panic within the program. The way to resolve it was to prevent the CPU counter from dropping below zero. Those were the bugs and design problems that were encountered during this project.

IX. Conclusion

During this final project, our main objective was to demonstrate a concurrent queue-based task dispatcher within Rust programming language. As you can see from this report and the code, it has been successfully done by me. The system is now a bounded worker pool with safe shared-state synchronization and measured scheduling behavior. Both the FIFO and optimized IO-priority schedulers were able to be tested in the same workload. However, the results were very similar. This showed that during the workload the resource constraints can be limited due to the impact of the scheduling changes. But overall, this project helped with the knowledge and understanding of what Rust is about during this semester and how it can be used for concurrency, synchronization, ownership and designing/analyzing concurrent systems.

